openapi: 3.1.0 info: title: AiFinPay Paywall Protocol (AIFP) API version: 1.0.0 summary: HTTP-402 native payment protocol for autonomous AI agents and merchants. description: | The **AiFinPay Paywall Protocol (AIFP)** is an open, HTTP-402-native payment standard that lets merchants charge AI agents per request and lets agents pay autonomously using stablecoins or hybrid fiat settlement.

```
This document is the **conforming API surface** for AIFP-1. The normative
protocol specification is **AIFP-1 (Doc 01)**. Where this document and AIFP-1
differ, **AIFP-1 governs**.

## Core flow
1. Agent hits a protected resource with quota exhausted → `402 Payment Required`
   with an AIFP Payment Challenge.
2. Agent requests a `POST /v1/quote`.
3. Agent pays via `POST /v1/pay` (idempotent) and receives an Ed25519 JWT receipt.
4. Agent retries the original request with `Payment-Receipt: <jwt>`. Merchant
   verifies the receipt **statelessly** and serves `200`.

## Pricing (canonical)
| Pricing Tier | Price (USD) |
|---|---|
| standard | USD 0.00001 |
| complex | USD 0.00006 |
| premium | USD 0.00010 |

Free quota: **100 requests** per agent per merchant per month.
Protocol fee: volume-tiered **1% / 1% / 1%** (cap ~1%).
```

termsOfService: https://aifinpay.io/terms contact: name: AiFinPay Protocol Team url: https://docs.aifinpay.io email: protocol@aifinpay.io license: name: Apache-2.0 url: https://www.apache.org/licenses/LICENSE-2.0 x-protocol: AIFP-1 x-protocol-version: "1.0.0"

servers: - url: https://api.aifinpay.io description: Production - url: https://sandbox.api.aifinpay.io description: Sandbox (test mode, no real settlement)

tags: - name: Quote description: Price discovery for a protected resource. - name: Pay description: Execute payment and obtain a receipt token. - name: Receipt description: Retrieve and inspect receipt status. - name: Verification description: Stateless and assisted receipt verification (JWKS). - name: Merchant description: Merchant onboarding, configuration, and resource pricing. - name: Wallet description: Agent wallet provisioning, funding, and budget policy. - name: Passport description: Agent Passport identity, reputation, and trust level. - name: Migration description: x402 compatibility and migration helpers. - name: Webhooks description: Asynchronous settlement and lifecycle events.

security: - ApiKeyAuth: []

paths: /v1/quote: post: tags: [Quote] operationId: createQuote summary: Create a price quote for a protected resource description: | Returns a binding price quote for a `(merchant, resource, pricing_tier)` tuple, including accepted assets/chains, a single-use `nonce`, and an `expires_at`. A quote is required before `POST /v1/pay`. security: - ApiKeyAuth: [] requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/QuoteRequest' } examples: standard: summary: Standard-pricing_tier data endpoint value: merchant_id: mrch_9f3a1c2b resource: /api/data

pricing_tier: standard currency: USD responses: '200': description: Quote issued. content: application/json: schema: { $ref: '#/components/schemas/Quote' } examples: standard: value: quote_id: qt_8d21f0 merchant_id: mrch_9f3a1c2b resource: /api/data amount: "0.00001" currency: USD accepted_assets: [USDC, USDT, PYUSD] accepted_chains: [polygon, base, solana] pay_to: { polygon: "0xMerchant...", solana: "Merc...111" } nonce: b7e2a1f3c91a4d6e8b0c2f4a6d8e0f2c expires_at: "2026-06-28T12:39:56Z" '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '404': { $ref: '#/components/responses/NotFound' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/pay: post: tags: [Pay] operationId: createPayment summary: Execute a payment for a quote and receive a receipt token description: | Settles a previously issued quote and returns an **Ed25519 JWT receipt**. MUST be sent with an `Idempotency-Key` header. Replaying the same key with an identical body returns the original receipt; a different body → `409`. Synchronous settlement returns `200`; async returns `202` with a poll URL. security: - ApiKeyAuth: [] parameters: - $ref: '#/components/parameters/IdempotencyKey' requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/PayRequest' } examples: pay_polygon: value: quote_id: qt_8d21f0 wallet_id: wlt_3a1b asset: USDC chain: polygon responses: '200': description: Settled synchronously; receipt issued. content: application/json: schema: { $ref: '#/components/schemas/Receipt' } examples: settled: value: receipt_id: rcpt_7b3e9f21 receipt: status: settled tx_ref: "0xabc123..." amount: "0.00001" fee: "0.0000001" expires_at: "2026-06-28T12:44:56Z" '202': description: Accepted; settlement is asynchronous. content: application/json: schema: { $ref: '#/components/schemas/SettlingReceipt' } examples: settling: value: receipt_id: rcpt_7b3e9f21 status: settling poll: /v1/receipt/rcpt_7b3e9f21 '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '403': { $ref: '#/components/responses/Forbidden' } '404': { $ref: '#/components/responses/NotFound' } '409': { $ref: '#/components/responses/Conflict' } '410': { $ref: '#/components/responses/Gone' } '422': { $ref: '#/components/responses/Unprocessable' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/receipt/{receipt_id}: get: tags: [Receipt] operationId: getReceipt summary: Retrieve a receipt and its settlement status description: | Returns the receipt record and current status. Used to poll after a `202` async settlement, or to audit a prior payment. security: - ApiKeyAuth: [] parameters: - name: receipt_id in: path required: true schema: { type: string, pattern: '^rcpt_[A-Za-z0-9]+$' } example: rcpt_7b3e9f21 responses: '200': description: Receipt record. content: application/json: schema: { $ref: '#/components/schemas/Receipt' } '404': { $ref: '#/components/responses/NotFound' }

/v1/verify: post: tags: [Verification] operationId: verifyReceipt summary: Assisted receipt verification (optional fallback) description: | **Conformant merchants MUST verify receipts locally and statelessly (AIFP-1 §7.4).** This endpoint is an OPTIONAL fallback ONLY for constrained edge proxies that cannot perform EdDSA verification locally (e.g., legacy NGINX/Apache without a native crypto module).

```
    Routine verification traffic MUST NOT be routed through this
    endpoint — doing so reintroduces a backend dependency and creates a
    DoS amplifier (AIFP Security Spec §16). This endpoint is subject to
    aggressive per-key rate limiting and returns `429` when the limit is
    exceeded. Merchants that use it SHOULD delegate to a local verifier
    sidecar instead (see Merchant Integration Guide §6.13–6.14).
```

```
    Performs the full AIFP-1 §7.4 algorithm (including the amount check)
    and returns the decision. The caller MUST supply the resource's
    current required price so the server can perform step 8.
  security:
    - ApiKeyAuth: []
  requestBody:
    required: true
    content:
      application/json:
        schema: { $ref: '#/components/schemas/VerifyRequest' }
  responses:
    '200':
      description: Verification decision.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/VerifyResult' }
    '400': { $ref: '#/components/responses/BadRequest' }
    '401': { $ref: '#/components/responses/Unauthorized' }
    '422': { $ref: '#/components/responses/Unprocessable' }
    '429': { $ref: '#/components/responses/RateLimited' }
```

/.well-known/jwks.json: get: tags: [Verification] operationId: getJwks summary: JSON Web Key Set for receipt verification description: | Public Ed25519 keys used to verify receipt signatures. Keyed by `kid` (e.g., `aifp-2026-06`). Cache per HTTP cache headers; rotate on `kid` change. security: [] responses: '200': description: JWKS. content: application/json: schema: { $ref: '#/components/schemas/JWKS' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/merchants: post: tags: [Merchant] operationId: createMerchant summary: Register a merchant account security: - ApiKeyAuth: [] requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/MerchantCreateRequest' } responses: '201': description: Merchant created. content: application/json: schema: { $ref: '#/components/schemas/Merchant' } '400': { $ref: '#/components/responses/BadRequest' }

/v1/merchants/{merchant_id}: get: tags: [Merchant] operationId: getMerchant summary: Retrieve a merchant security: - ApiKeyAuth: [] parameters: - $ref: '#/components/parameters/MerchantId' responses: '200': description: Merchant record. content: application/json: schema: { $ref: '#/components/schemas/Merchant' } '404': { $ref: '#/components/responses/NotFound' }

/v1/merchants/{merchant_id}/resources: put: tags: [Merchant] operationId: setMerchantResources summary: Configure resource pricing for a merchant description: Map resource paths to pricing_tier tiers (standard/complex/premium). security: - ApiKeyAuth: [] parameters: - $ref: '#/components/parameters/MerchantId' requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/ResourcePricingMap' } responses: '200': description: Updated pricing map. content: application/json: schema: { $ref: '#/components/schemas/ResourcePricingMap' } '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '404': { $ref: '#/components/responses/NotFound' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/wallets: post: tags: [Wallet] operationId: createWallet summary: Provision an agent wallet security: - ApiKeyAuth: [] requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/WalletCreateRequest' } responses: '201': description: Wallet created. content: application/json: schema: { $ref: '#/components/schemas/Wallet' } '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '409': { $ref: '#/components/responses/Conflict' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/wallets/{wallet_id}/budget: put: tags: [Wallet] operationId: setWalletBudget summary: Set or update a wallet budget policy description: | Budget policies cap spend per time window and per merchant. Exceeding a policy at pay time yields `403 AIFP-403-BUDGET-EXCEEDED`. security: - ApiKeyAuth: [] parameters: - $ref: '#/components/parameters/WalletId' requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/BudgetPolicy' } responses: '200': description: Updated budget policy. content: application/json: schema: { $ref: '#/components/schemas/BudgetPolicy' } '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '403': { $ref: '#/components/responses/Forbidden' } '404': { $ref: '#/components/responses/NotFound' } '422': { $ref: '#/components/responses/Unprocessable' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/passports: post: tags: [Passport] operationId: createPassport summary: Issue an Agent Passport description: | Creates an Agent Passport ( `pp_*` ) bound to an agent ( `agt_*` ), with an Ed25519 identity key, reputation (start 500), and trust level. security: - ApiKeyAuth: [] requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/PassportCreateRequest' } responses: '201': description: Passport issued. content: application/json: schema: { $ref: '#/components/schemas/Passport' } '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '409': { $ref: '#/components/responses/Conflict' } '422': { $ref: '#/components/responses/Unprocessable' } '429': { $ref: '#/components/responses/RateLimited' }

/v1/passports/{passport_id}: get: tags: [Passport] operationId: getPassport summary: Retrieve an Agent Passport (reputation, risk, trust) security: - ApiKeyAuth: [] parameters: - name: passport_id in: path required: true schema: { type: string, pattern: '^pp_[A-Za-z0-9]+$' } responses: '200': description: Passport record. content: application/json: schema: { $ref: '#/components/schemas/Passport' } '404': { $ref: '#/components/responses/NotFound' }

/v1/migrate/x402: post: tags: [Migration] operationId: migrateFromX402 summary: Migrate an x402 integration to AIFP description: | Bridges an existing x402 client into an AIFP agent identity. Maps x402 header challenges to AIFP Payment Challenges. security: - ApiKeyAuth: [] requestBody: required: true content: application/json: schema: { $ref: '#/components/schemas/X402MigrationRequest' } responses: '200': description: Migration context established. content: application/json: schema: { $ref: '#/components/schemas/X402MigrationResult' } '400': { $ref: '#/components/responses/BadRequest' } '401': { $ref: '#/components/responses/Unauthorized' } '404': { $ref: '#/components/responses/NotFound' } '422': { $ref: '#/components/responses/Unprocessable' } '429': { $ref: '#/components/responses/RateLimited' }

components: securitySchemes: ApiKeyAuth: type: apiKey in: header name: Authorization description: | Bearer API key. Use `Authorization: Bearer sk_live_...` (production) or `sk_test_...` (sandbox). Agent identity is additionally asserted via the Agent Passport signature (AIFP-1 §10).

parameters: IdempotencyKey: name: Idempotency-Key in: header required: true schema: { type: string, format: uuid } description: | Client-generated UUID. MUST be drawn from a CSPRNG with ≥ 122 bits of entropy (e.g., `crypto.randomUUID()`, `secrets.token_hex(16)`, `os.urandom(16)`). DO NOT use `Math.random()` or time-based seeds — weak RNG enables forced-collision denial (AIFP-1 §8.5 / Security Spec §10). Dedupe window 24h. MerchantId: name: merchant_id in: path required: true schema: { type: string, pattern: '^mrch_[A-Za-z0-9]+$' } WalletId: name: wallet_id in: path required: true schema: { type: string, pattern: '^wlt_[A-Za-z0-9]+$' }

responses: BadRequest: description: Malformed request (AIFP-400). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: bad_request, code: AIFP-400, message: "Invalid JSON / missing field", request_id: req_8f12, doc: "https://docs.aifinpay.io/errors/AIFP-400" } } Unauthorized: description: Missing/invalid API key (AIFP-401). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: unauthorized, code: AIFP-401, message: "Missing or invalid API key", request_id: req_8f13, doc: "https://docs.aifinpay.io/errors/AIFP-401" } } Forbidden: description: Blocklist / KYC / budget (AIFP-403, AIFP-403-BUDGET-EXCEEDED). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: forbidden, code: AIFP-403-BUDGET-EXCEEDED, message: "Payment would breach budget policy", request_id: req_8f14, doc: "https://docs.aifinpay.io/errors/AIFP-403-BUDGET-EXCEEDED" } } NotFound: description: Quote/receipt/merchant not found (AIFP-404). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: not_found, code: AIFP-404, message: "Resource not found", request_id: req_8f15, doc: "https://docs.aifinpay.io/errors/AIFP-404" } } Conflict: description: Receipt replay / idempotency conflict (AIFP-409). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: conflict, code: AIFP-409, message: "Receipt replay or idempotency conflict", request_id: req_8f16, doc: "https://docs.aifinpay.io/errors/AIFP-409" } } Gone: description: Quote expired (AIFP-410). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: gone, code: AIFP-410, message: "Quote expired", request_id: req_8f17, doc: "https://docs.aifinpay.io/errors/AIFP-410" } } Unprocessable: description: Amount mismatch / bad signature (AIFP-422). content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: invalid_receipt, code: AIFP-422-SIGNATURE, message: "Receipt signature verification failed.", request_id: req_8f18, doc: "https://docs.aifinpay.io/errors/AIFP-422-SIGNATURE" } } RateLimited: description: Rate limited (AIFP-429). Honor `Retry-After`. headers: Retry-After: { schema: { type: integer }, description: Seconds to wait. } content: application/json: schema: { $ref: '#/components/schemas/Error' } example: { error: { type: rate_limited, code: AIFP-429, message: "Request limit exceeded", request_id: req_8f19, doc: "https://docs.aifinpay.io/errors/AIFP-429" } }

schemas: PricingTier: type: string enum: [standard, complex, premium] description: | Pricing tier. standard=USD 0.00001, complex=USD 0.00006, premium=USD 0.00010. Asset: type: string enum: [USDC, USDT, PYUSD] Chain: type: string description: | One of the 12 supported networks. Full Core (8): solana, polygon, avalanche, bnb, optimism, arbitrum, base, unichain. Splitter-only EVM (2): bot_chain, xrpl_evm. Splitter MVP non-EVM (2): near, aptos. enum: [solana, polygon, avalanche, bnb, optimism, arbitrum, base, unichain, bot_chain, xrpl_evm, near, aptos]

```
QuoteRequest:
  type: object
```

```
  required: [merchant_id, resource, pricing_tier]
  properties:
    merchant_id: { type: string, pattern: '^mrch_[A-Za-z0-9]+$' }
    resource: { type: string, example: /api/data }
    pricing_tier: { $ref: '#/components/schemas/PricingTier' }
    currency: { type: string, default: USD, example: USD }
Quote:
  type: object
  required: [quote_id, merchant_id, resource, amount, currency, nonce, expires_at]
  properties:
    quote_id: { type: string, pattern: '^qt_[A-Za-z0-9]+$' }
    merchant_id: { type: string }
    resource: { type: string }
    amount: { type: string, description: "Decimal string in the quote currency.", examp
    currency: { type: string, example: USD }
    accepted_assets:
      type: array
      items: { $ref: '#/components/schemas/Asset' }
    accepted_chains:
      type: array
      items: { $ref: '#/components/schemas/Chain' }
    pay_to:
      type: object
      additionalProperties: { type: string }
      description: Map of chain → settlement address.
    nonce: { type: string, minLength: 22, maxLength: 128, pattern: '^[A-Za-z0-9_-]+$',
    expires_at: { type: string, format: date-time }

PayRequest:
  type: object
  required: [quote_id, wallet_id, asset, chain]
  properties:
    quote_id: { type: string, pattern: '^qt_[A-Za-z0-9]+$' }
    wallet_id: { type: string, pattern: '^wlt_[A-Za-z0-9]+$' }
    asset: { $ref: '#/components/schemas/Asset' }
    chain: { $ref: '#/components/schemas/Chain' }
Receipt:
  type: object
  required: [receipt_id, status]
  properties:
    receipt_id: { type: string, pattern: '^rcpt_[A-Za-z0-9]+$' }
    receipt: { type: string, description: Compact Ed25519 JWT receipt token. }
    status: { type: string, enum: [settled, settling, expired, revoked] }
    tx_ref: { type: string, description: On-chain tx hash or fiat reference. }
    amount: { type: string, example: "0.00001" }
    fee: { type: string, example: "0.0000001" }
    expires_at: { type: string, format: date-time }
SettlingReceipt:
  type: object
  required: [receipt_id, status, poll]
  properties:
    receipt_id: { type: string }
    status: { type: string, enum: [settling] }
    poll: { type: string, example: /v1/receipt/rcpt_7b3e9f21 }
```

```
ReceiptClaims:
  type: object
  description: Decoded JWT claim set (AIFP-1 §7.3).
  required: [iss, sub, aud, resource, amount, nonce, iat, exp, kid]
  properties:
    iss: { type: string, const: https://api.aifinpay.io }
    sub: { type: string, pattern: '^agt_[A-Za-z0-9]+$' }
    aud: { type: string, pattern: '^mrch_[A-Za-z0-9]+$' }
    resource: { type: string }
    pricing_tier: { $ref: '#/components/schemas/PricingTier' }
    amount: { type: string }
    currency: { type: string }
    asset: { $ref: '#/components/schemas/Asset' }
    chain: { $ref: '#/components/schemas/Chain' }
    tx_ref: { type: string }
    receipt_id: { type: string }
    nonce: { type: string, minLength: 22, maxLength: 128, pattern: '^[A-Za-z0-9_-]+$',
    iat: { type: integer }
    exp: { type: integer, description: TTL default 600s. }
    kid: { type: string, example: aifp-2026-06 }
    quota: { type: integer, minimum: 1, description: "Optional multi-use claim (AIFP-1

VerifyRequest:
  type: object
  required: [receipt, merchant_id, resource, required_amount]
  properties:
    receipt: { type: string, description: Compact JWT. }
    merchant_id: { type: string }
    resource: { type: string }
    required_amount:
      type: string
      description: |
        The resource's current required price (decimal USD string). The
        server performs the full AIFP-1 §7.4 step-8 amount check
        (`amount >= required_amount`). Without this value the amount
        check cannot be performed and a low-amount receipt could pass.
VerifyResult:
  type: object
  required: [valid]
  properties:
    valid: { type: boolean }
    reason: { type: string, description: Present when valid=false. }
    claims: { $ref: '#/components/schemas/ReceiptClaims' }

JWKS:
  type: object
  required: [keys]
  properties:
    keys:
      type: array
      items:
        type: object
        required: [kty, crv, x, kid, use, alg]
        properties:
          kty: { type: string, const: OKP }
```

```
          crv: { type: string, const: Ed25519 }
          x: { type: string, description: Base64url public key. }
          kid: { type: string, example: aifp-2026-06 }
          use: { type: string, const: sig }
          alg: { type: string, const: EdDSA }

MerchantCreateRequest:
  type: object
  required: [name]
  properties:
    name: { type: string }
    settlement:
      type: object
      properties:
        mode: { type: string, enum: [stablecoin, fiat, hybrid], default: stablecoin }
        payout_addresses:
          type: object
          additionalProperties: { type: string }
Merchant:
  type: object
  required: [merchant_id, name, created_at]
  properties:
    merchant_id: { type: string, pattern: '^mrch_[A-Za-z0-9]+$' }
    name: { type: string }
    settlement_mode: { type: string, enum: [stablecoin, fiat, hybrid] }
    fee_tier:
      type: integer
      enum: [1, 2, 3]
      description: Protocol fee tier. Lower values correspond to lower protocol fees; t
    created_at: { type: string, format: date-time }
ResourcePricingMap:
  type: object
  additionalProperties: { $ref: '#/components/schemas/PricingTier' }
  example: { /api/data: standard, /api/search: complex, /api/llm: premium }

WalletCreateRequest:
  type: object
  required: [agent_id, type]
  properties:
    agent_id: { type: string, pattern: '^agt_[A-Za-z0-9]+$' }
    type: { type: string, enum: [custodial, non_custodial], default: non_custodial }
Wallet:
  type: object
  required: [wallet_id, agent_id, type]
  properties:
    wallet_id: { type: string, pattern: '^wlt_[A-Za-z0-9]+$' }
    agent_id: { type: string }
    type: { type: string, enum: [custodial, non_custodial] }
    balances:
      type: object
      additionalProperties: { type: string }
BudgetPolicy:
  type: object
  properties:
    per_window:
```

```
      type: object
      properties:
        window: { type: string, enum: [hour, day, week, month] }
        cap_usd: { type: string, example: "50.00" }
    per_merchant:
      type: object
      additionalProperties: { type: string }
      description: Map merchant_id → cap_usd.

PassportCreateRequest:
  type: object
  required: [agent_id]
  properties:
    agent_id: { type: string, pattern: '^agt_[A-Za-z0-9]+$' }
    public_key: { type: string, description: Ed25519 public key (base64url). }
Passport:
  type: object
  required: [passport_id, agent_id, reputation, risk, trust_level]
  properties:
    passport_id: { type: string, pattern: '^pp_[A-Za-z0-9]+$' }
    agent_id: { type: string, pattern: '^agt_[A-Za-z0-9]+$' }
    public_key: { type: string }
    reputation: { type: integer, minimum: 0, maximum: 1000, default: 500 }
    risk: { type: integer, minimum: 0, maximum: 100 }
    trust_level: { type: string, enum: [untrusted, basic, verified, enterprise] }
    created_at: { type: string, format: date-time }

X402MigrationRequest:
  type: object
  required: [agent_id]
  properties:
    agent_id: { type: string }
    x402_endpoint: { type: string, format: uri }
X402MigrationResult:
  type: object
  properties:
    migration_id: { type: string }
    migration_program: { type: string, description: "Optional program metadata, if a pu
    status: { type: string, enum: [active] }

Error:
  type: object
  required: [error]
  properties:
    error:
      type: object
      required: [type, code, message]
      properties:
        type: { type: string, example: invalid_receipt }
        code: { type: string, example: AIFP-422-SIGNATURE }
        message: { type: string }
        request_id: { type: string, example: req_8f12 }
        doc: { type: string, format: uri }
```